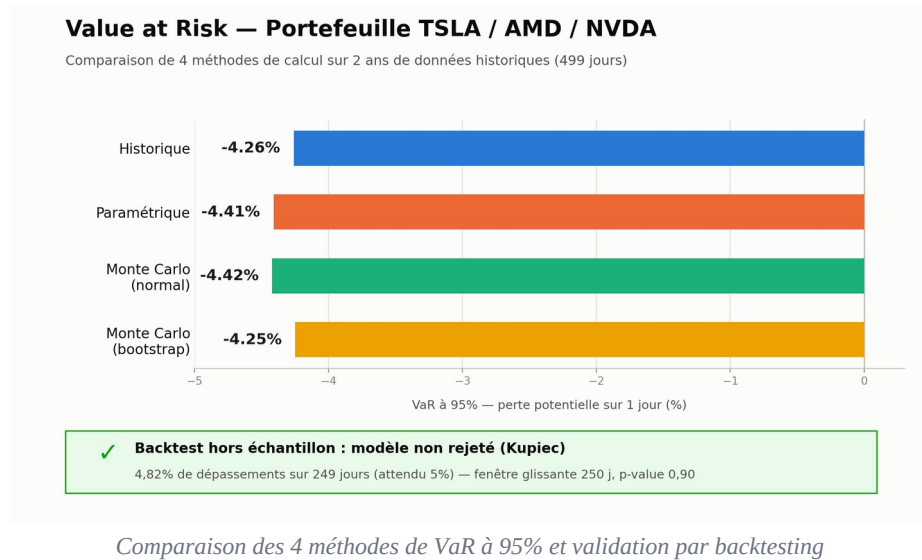


Analyse de Risque de Portefeuille

Calcul et validation de la Value at Risk (VaR)

Projet personnel — recherche de stage / alternance en Front Office, Gestion des Risques, Produits Structurés



1. Contexte et objectif

Ce projet complète un premier travail d'automatisation de la réconciliation post-trade en abordant un autre pilier du front office : la mesure du risque de marché. L'objectif est d'implémenter, comparer et valider plusieurs méthodes de calcul de la Value at Risk (VaR) — l'indicateur standard utilisé par les desks pour quantifier la perte potentielle d'un portefeuille.

Le portefeuille étudié est composé de trois valeurs technologiques — Tesla (TSLA), AMD et Nvidia (NVDA) —, équipondérées, sur deux ans de données réelles (27 septembre 2024 au 27 septembre 2026, 499 jours de bourse). Ce choix, délibérément volatil, permet d'obtenir des résultats de VaR significatifs et de bien illustrer chaque méthode.

2. Méthodologie

2.1 Récupération des données et calcul des rendements

Les prix de clôture sont récupérés via l'API Yahoo Finance, puis convertis en rendements journaliers : $(\text{Prix du jour} - \text{Prix de la veille}) / \text{Prix de la veille}$, exprimé en pourcentage. Ce choix rend les actions comparables entre elles (une variation de 5\$ n'a pas le même poids selon le niveau du prix) et donne une série stationnaire, contrairement aux prix bruts qui suivent une tendance — c'est cette propriété qui permet de calculer une moyenne, un écart-type et une VaR.

```
!pip install yfinance -q
import yfinance as yf
import pandas as pd
import numpy as np

data = yf.download(["TSLA", "AMD", "NVDA"], start="2024-09-27", end="2026-09-27")
close_prices = data["Close"]
```

```
returns = close_prices.pct_change().dropna()
returns.describe()
```

2.2 Construction du portefeuille

Le portefeuille est construit avec une pondération égale entre les trois actions (1/3 chacune), rééquilibrée quotidiennement par hypothèse simplificatrice.

```
weights = np.array([1/3, 1/3, 1/3])
portfolio_returns = returns.dot(weights)
```

2.3 VaR historique

La VaR historique prend directement le 5e percentile des rendements observés du portefeuille : le seuil en dessous duquel se trouvent les 5% pires jours de l'historique, sans hypothèse sur la forme de la distribution.

```
var_95 = np.percentile(portfolio_returns, 5)
print(f"VaR 95% (1 jour) : {var_95:.2%}")
```

2.4 VaR paramétrique

Cette méthode suppose que les rendements suivent une loi normale, et calcule le seuil à partir de la moyenne (μ) et de l'écart-type (σ) :

$$\text{VaR} = \mu + z \times \sigma \quad (z \approx -1,645 \text{ pour un seuil à 95\%})$$

```
from scipy.stats import norm

mu = portfolio_returns.mean()
sigma = portfolio_returns.std()
z = norm.ppf(0.05)
var_95_param = mu + z * sigma
print(f"VaR paramétrique 95% (1 jour) : {var_95_param:.2%}")
```

2.5 Comparaison à 99% (effet des queues épaisses)

À 95%, l'écart entre VaR historique et paramétrique reste faible ; on vérifie s'il se creuse à un seuil plus extrême (99%), là où l'hypothèse de loi normale est la plus susceptible d'être prise en défaut.

```
var_99_hist = np.percentile(portfolio_returns, 1)
var_99_param = mu + norm.ppf(0.01) * sigma
print(f"VaR 99% historique : {var_99_hist:.2%}")
print(f"VaR 99% paramétrique : {var_99_param:.2%}")
```

2.6 VaR Monte Carlo

100 000 scénarios de rendements sont simulés à partir de la moyenne et de la matrice de covariance des 3 actions (qui capture leur dépendance), puis on prend le 5e percentile des rendements de portefeuille simulés. Une variante bootstrap ré-échantillonne directement parmi les vrais rendements observés plutôt que de simuler depuis une loi théorique, ce qui capture mieux les pertes extrêmes réelles.

```
mean_vec = returns.mean().values
cov_matrix = returns.cov().values
```

```

np.random.seed(42)
sim_returns = np.random.multivariate_normal(mean_vec, cov_matrix, 100_000)
sim_portfolio = sim_returns.dot(weights)
var_95_mc = np.percentile(sim_portfolio, 5)

sampled_returns = returns.sample(n=100_000, replace=True).values
sim_portfolio_boot = sampled_returns.dot(weights)
var_95_boot = np.percentile(sim_portfolio_boot, 5)

```

2.7 Backtesting initial (en échantillon) et sa limite

Un premier backtest compte le nombre de fois où le rendement réel a dépassé le seuil de VaR historique à 95%, sur les 499 jours.

```

breaches = portfolio_returns < var_95
breach_rate = breaches.sum() / len(portfolio_returns)
print(f"Dépassements : {breaches.sum()} sur {len(portfolio_returns)} jours
      ({breach_rate:.2%})")

```

Ce test est circulaire : la VaR historique est le 5e percentile des 499 rendements, puis on la compare à ces mêmes 499 rendements. On trouve donc environ 5% de dépassements (25 sur 499) par construction, quelle que soit la qualité du modèle. Il ne valide rien ; il sert ici d'illustration graphique (partie 4), et la validation réelle est faite hors échantillon ci-dessous.

2.8 Backtesting hors échantillon (fenêtre glissante)

Pour tester réellement le modèle, la VaR du jour t est calculée uniquement avec les 250 derniers jours connus (environ un an de bourse), puis comparée au rendement réel du jour t , que le modèle n'a jamais vu. Le décalage d'un jour (`shift(1)`) évite tout biais de look-ahead. Il reste 249 jours de test ($499 - 250$).

```

window = 250
for conf in [0.95, 0.99]:
    p = 1 - conf
    var_hist_roll = portfolio_returns.rolling(window).quantile(p).shift(1)
    var_param_roll = (portfolio_returns.rolling(window).mean()
                     + norm.ppf(p) * portfolio_returns.rolling(window).std()).shift(1)
    for name, var_roll in [("Historique", var_hist_roll), ("Paramétrique",
var_param_roll)]:
        valid = var_roll.notna()
        breaches = portfolio_returns[valid] < var_roll[valid]
        results[(name, conf)] = (valid.sum(), breaches.sum())

```

2.9 Test de Kupiec

Le test de Kupiec (proportion of failures) vérifie si le taux de dépassement observé est statistiquement compatible avec le taux attendu p . Il compare la vraisemblance des données sous p à celle obtenue avec le taux observé x/n : $LR = -2 \times [\ln L(p) - \ln L(x/n)]$, qui suit une loi du khi-deux à 1 degré de liberté sous l'hypothèse que le modèle est correct. Le modèle est rejeté si $LR > 3,84$ ($p\text{-value} < 5\%$).

```

def kupiec_test(x, n, p):
    taux_obs = x / n
    ll_h0 = (n - x) * np.log(1 - p) + x * np.log(p)
    ll_h1 = (n - x) * np.log(1 - taux_obs) + x * np.log(taux_obs) if x > 0 else n *
np.log(1 - taux_obs)
    lr = -2 * (ll_h0 - ll_h1)
    return lr, 1 - chi2.cdf(lr, df=1)

```

3. Résultats

Méthode	VaR 95% (1j)	VaR 99% (1j)
Historique	-4.26%	-7.40%
Paramétrique	-4.41%	-6.32%
Monte Carlo (normal)	-4.42%	—
Monte Carlo (bootstrap)	-4.25%	—

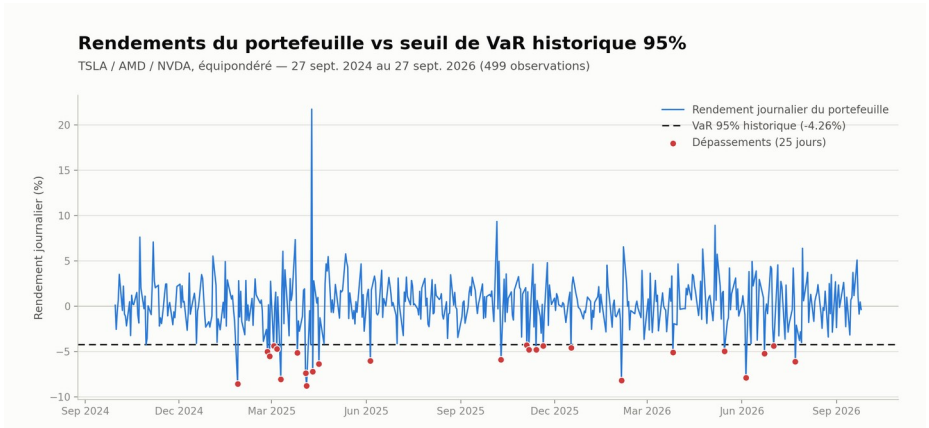
Backtesting hors échantillon (fenêtre glissante de 250 jours, 249 jours testés) :

Méthode	Confiance	Dépassements	Taux observé	LR	p-value	Verdict
Historique	95%	12 / 249	4,82%	0,017	0,895	Non rejeté
Paramétrique	95%	12 / 249	4,82%	0,017	0,895	Non rejeté
Historique	99%	3 / 249	1,20%	0,099	0,753	Non rejeté
Paramétrique	99%	3 / 249	1,20%	0,099	0,753	Non rejeté

Lecture : hors échantillon, les taux de dépassement (4,82% et 1,20%) sont proches des taux attendus (5% et 1%) et le test de Kupiec ne rejette le modèle dans aucun cas. Le premier backtest en échantillon (25 dépassements sur 499 jours, 5,01%) n'avait pas de valeur de validation, puisqu'il est vrai par construction. Avec 249 jours de test, la puissance du test reste limitée, surtout à 99% où l'on attend environ 2 à 3 dépassements : « non rejeté » signifie « pas de preuve que le modèle est mauvais », pas « modèle prouvé ». Les deux méthodes ont ici le même nombre de dépassements, ce qui tient à l'échantillon ; elles diffèrent surtout par le niveau de la VaR (voir ci-dessus).

Les quatre méthodes convergent à 95% (écart maximal de 0,2 point de pourcentage), ce qui valide la cohérence des implémentations. L'écart se creuse en revanche nettement à 99% (-7,40% contre -6,32%), illustrant une limite bien connue de l'hypothèse de loi normale : elle sous-estime la fréquence des pertes extrêmes ("queues épaisses", ou fat tails).

4. Analyse graphique



Rendements journaliers du portefeuille, seuil de VaR historique et 25 dépassements (illustration en échantillon)

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 6))
plt.plot(portfolio_returns.index, portfolio_returns, label="Rendements du portefeuille",
color="steelblue")
plt.axhline(y=var_95, color="red", linestyle="--", label="VaR 95% historique")
plt.scatter(portfolio_returns.index[breaches], portfolio_returns[breaches], color="red",
zorder=5, label="Dépassements")
plt.legend()
plt.show()
```

Ce graphique appelle une observation supplémentaire : les dépassements ne sont pas répartis uniformément dans le temps, mais concentrés sur deux périodes (début 2025 et fin 2025/début 2026). Ce phénomène, appelé clustering de volatilité, montre que le risque n'est pas constant dans le temps — une limite du calcul de VaR statique présenté ici, qui ne capture pas les phases où le marché devient plus ou moins turbulent.

5. Limites et pistes d'amélioration

- La VaR ne renseigne pas sur l'ampleur des pertes au-delà du seuil — l'Expected Shortfall (moyenne des pertes dans les 5% pires cas) serait un complément naturel.
- L'hypothèse de volatilité constante ne capture pas le clustering observé sur le graphique — un modèle GARCH permettrait de faire varier la VaR dans le temps selon le régime de volatilité.
- Le test de Kupiec ne vérifie que le nombre de dépassements, pas leur répartition dans le temps. Le test de Christoffersen (indépendance des dépassements) serait le complément naturel face au clustering observé.
- Avec 249 jours de test, la puissance statistique est limitée, surtout à 99% : un backtest sur une période plus longue, incluant plusieurs régimes de marché, renforcerait la conclusion.
- Le portefeuille suppose un rééquilibrage quotidien parfait à poids égaux, une simplification par rapport à un portefeuille réel.
- Le choix de 3 valeurs technologiques très corrélées entre elles limite l'effet de diversification (voir corrélations en annexe) — un portefeuille plus diversifié donnerait une VaR relative plus faible.

6. Conclusion

Ce projet démontre la capacité à implémenter plusieurs méthodes de calcul de VaR reconnues (historique, paramétrique, Monte Carlo), à les comparer de façon critique, et à les tester hors échantillon (fenêtre glissante de 250 jours et test de Kupiec) — une démarche proche de ce qu'attend une fonction risque sur un desk. Le projet a aussi mis en évidence, puis corrigé, une limite de méthode : un premier backtest en échantillon, circulaire par construction, ne validait rien. Le code Python de chaque méthode est présenté dans la partie 2, et le notebook complet est disponible en ligne.

Annexe A — Données utilisées

A.1 Source et périmètre

Les prix de clôture (ajustés des splits et dividendes) ont été récupérés via l'API Yahoo Finance (bibliothèque Python yfinance), pour les tickers TSLA, AMD et NVDA, du 27 septembre 2024 au 27 septembre 2026 (499 rendements journaliers exploitables après retrait du premier jour sans historique).

A.2 Échantillon de prix de clôture

Date	AMD (\$)	NVDA (\$)	TSLA (\$)
2024-09-27	164.35	121.08	260.46
2024-09-30	164.08	121.12	261.63
2024-10-01	159.75	116.69	258.02
...	...	...	...
2026-09-23	614.61	225.51	380.12
2026-09-24	629.26	224.58	377.94
2026-09-25	630.63	225.07	372.11

Sur la période, AMD est passé de 164\$ à 631\$ (+284%), portée par le cycle haussier des semi-conducteurs — un contexte à garder en tête pour interpréter les statistiques ci-dessous.

A.3 Statistiques descriptives des rendements journaliers

Statistique	AMD	NVDA	TSLA
Observations	499	499	499
Moyenne	0.347%	0.163%	0.142%
Écart-type	3.989%	2.781%	3.770%
Minimum	-17.31%	-16.97%	-15.43%
Médiane	0.151%	0.237%	0.016%
Maximum	23.82%	18.72%	22.69%
Volatilité annualisée	63.3%	44.1%	59.8%

La volatilité annualisée (écart-type journalier $\times \sqrt{252}$) permet de comparer directement au marché : à titre de repère, le S&P 500 affiche typiquement une volatilité annualisée de 15 à 20%, contre 44 à 63% ici — cela confirme le choix délibéré d'un portefeuille à forte volatilité pour ce projet.

A.4 Matrice de corrélation des rendements

	AMD	NVDA	TSLA
AMD	1.00	0.53	0.42
NVDA	0.53	1.00	0.43
TSLA	0.42	0.43	1.00

Les corrélations modérées (0,42 à 0,53) montrent que les trois actions ne bougent pas de façon totalement indépendante — c'est cette information qui alimente la matrice de covariance utilisée dans la VaR Monte Carlo. Une corrélation plus faible entre les positions réduirait la VaR du portefeuille par effet de diversification.

A.5 Outils et reproductibilité

Python (pandas, numpy, scipy.stats, matplotlib), notebook Google Colab. Le code de chaque méthode est présenté directement sous sa section correspondante en partie 2 ; le lien du notebook exécutable complet est fourni avec ce document.